A03 - Vishvamitra: The Broken Invocation

Category: Crypto (Chaum-Pedersen / Fiat-Shamir) | **Flag:** `PINAKA{vishvamitra_bala_atibala_one_syllable_unbound}`

TL;DR

The armory accepts Chaum-Pedersen proofs made non-interactive with Fiat-Shamir, but the challenge hash covers only the commitments `(t1, t2)` and never the statement `(A, B)`. The forbidden Brahmastra tuple is the public Bala-Atibala tuple shifted by a known offset `d`, so I kept the leaked proof's commitments and moved the response to `s' = s + c*d mod q`. That proof verifies for Brahmastra without knowing any witness.

Recon

The challenge ships `params.json`, `instance.json`, the public protocol module `pinaka_invocation.py`, and a README. The module's own self-test confirms the leaked grant really verifies for `bala-atibala`. `params.json` gives the elevation offset `d` and states the Fiat-Shamir rule outright: `c = SHA256('PINAKA-A03-FS|t1|<t1>|t2|<t2>') mod q`. The target is a valid proof for `brahmastra`, posted to `/armory/invoke`.

```
A03_player ❯ ls -lgoh && python pinaka_invocation.py && jq '{elevation_offset_d, fiat_shamir}' params.json &&
 jq '.target' instance.json
total 28K
-rw-rw-r-- 1 3.3K Oct 30 01:43 instance.json
-rw-rw-r-- 1 1.6K Oct 30 01:43 params.json
-rw-r--r-- 1  981 Nov  1 20:00 payload.json
-rw-rw-r-- 1 3.0K Oct 30 01:43 pinaka_invocation.py
drwxr-xr-x 1   66 Nov  1 19:58 __pycache__
-rw-rw-r-- 1 2.4K Oct 30 01:46 README.md
drwxr-xr-x 1  132 Nov  1 19:59 screenshots
-rw-r--r-- 1 1.9K Nov  1 19:58 solve.py
-rw-r--r-- 1  777 Nov  1 19:58 verify.py
leaked grant for 'bala-atibala' verifies: True
{
  "elevation_offset_d": 10688494437297532970000848570232076811277444174337591363787724193449772577032
2,
  "fiat_shamir": "SHIPPED challenge c = SHA256('PINAKA-A03-FS|t1|'<t1>'|t2|'<t2>) mod q (decimal string encod
ing of t1,t2). See player/pinaka_invocation.py."
}
{
  "forbidden_weapon": "brahmastra",
  "submit_to": "POST /armory/invoke",
  "submit_format": {
    "weapon": "brahmastra",
    "proof": {
      "t1": "<int>",
      "t2": "<int>",
      "s": "<int>"
    }
  },
  "win_condition": "A valid Chaum-Pedersen proof for the brahmastra boon tuple releases the flag."
}

A03_player ❯
```

The group is the order-`q` subgroup of `Z_p*` with a 1024-bit safe prime `p = 2q + 1`. A statement `(A, B)` is true when one witness `w` gives `A = g^w` and `B = h^w`, and the verifier accepts `(t1, t2, s)` when `g^s = t1 * A^c` and `h^s = t2 * B^c (mod p)`.

The unbound challenge

Reading `fiat_shamir_challenge` and `verify` side by side shows the bug. The hash takes `t1` and `t2` only. `A` and `B` are used in the verification equations but never reach the hash, so the challenge `c` is the same no matter which statement a proof is checked against.

```
A03_player ❯ sed -n '/^def fiat_shamir_challenge/,/^$/p;/^def verify/,/^$/p' pinaka_invocation.py
def fiat_shamir_challenge(t1, t2, q):
    m = hashlib.sha256(b"PINAKA-A03-FS")
    m.update(b"|t1|"); m.update(str(t1).encode())
    m.update(b"|t2|"); m.update(str(t2).encode())
    return int(m.hexdigest(), 16) % q

def verify(params, A, B, proof):
    """Verify a Chaum-Pedersen proof (t1,t2,s) for boon tuple (A,B). Returns bool."""
    p, g, h, q = params["p"], params["g"], params["h"], params["q"]
    t1, t2, s = int(proof["t1"]), int(proof["t2"]), int(proof["s"])
    A, B = int(A), int(B)
    c = fiat_shamir_challenge(t1, t2, q)
    eq1 = pow(g, s, p) == (t1 * pow(A, c, p)) % p
    eq2 = pow(h, s, p) == (t2 * pow(B, c, p)) % p
    return eq1 and eq2


A03_player ❯
```

That by itself doesn't give a forgery, since I still need a valid `s` for the new statement. The elevation rule supplies it: `A1 = A0 * g^d` and `B1 = B0 * h^d` with the same public `d`, so the Brahmastra witness is `w0 + d`. The honest response is `s = r + c*w0`, so `s + c*d = r + c*(w0 + d)` is exactly the honest response for Brahmastra, and I never need `r` or `w0`.

Checking the `g` side (the `h` side is identical):

```
g^(s + c*d) = g^s * (g^d)^c = t1 * A0^c * (g^d)^c = t1 * (A0 * g^d)^c = t1 * A1^c
```

Forging

`solve.py` checks the elevation relation, confirms the leaked proof, computes `c` from the unchanged commitments, shifts the response, verifies the result locally against the Brahmastra tuple, and writes `payload.json`. With `--offline` it stops before submitting.

```
A03_player ❯ python solve.py --offline
[+] elevation holds: A1 = A0*g^d, B1 = B0*h^d
[+] leaked proof verifies for bala-atibala
[*] d  = 106884944372975329700008485702320768112774441743375913637877241934497725770322
[*] c  = ...ec4e2da8180a72c9d8f28152
[*] s  = ...3e9b78abc833f1f8ffd9dd56
[*] s' = ...739cfe7d60b6f87fc7e13d9a  (s + c*d mod q)
[+] forged proof verifies for brahmastra
[+] wrote payload.json

A03_player ❯
```

```
from pinaka_invocation import fiat_shamir_challenge, load_instance, load_params, verify

P, I = load_params(), load_instance()
p, q, g, h, d = P["p"], P["q"], P["g"], P["h"], P["elevation_offset_d"]
lo = I["weapons"]["bala-atibala"]["boon_tuple"]
hi = I["weapons"]["brahmastra"]["boon_tuple"]
```

```
pf = I["leaked_grant"]["proof"]

assert hi["A"] == lo["A"] * pow(g, d, p) % p
assert hi["B"] == lo["B"] * pow(h, d, p) % p
assert verify(P, lo["A"], lo["B"], pf)

c = fiat_shamir_challenge(pf["t1"], pf["t2"], q)
forged = {"t1": pf["t1"], "t2": pf["t2"], "s": (pf["s"] + c * d) % q}
assert verify(P, hi["A"], hi["B"], forged)
```

Sanity check

`verify.py` runs both proofs through the armory's own `verify()`. The leaked proof only verifies for Bala-Atibala and the forged one only verifies for Brahmastra, and both share the same challenge because `A` and `B` never enter the hash.

```
A03_player ❯ python verify.py
statement      leaked proof   forged proof
bala-atibala   True           False
brahmastra     False          True

same challenge for both proofs: True  (A, B never enter the hash)

A03_player ❯
```

Flag

Posting `payload.json` to the validator returns the flag.

```
A03_player ❯ curl -s -X POST https://pinaka-a03-cyber-xo-x.vercel.app/armory/invoke -H 'Content-Type: applica
tion/json' -d @payload.json | jq
{
  "flag": "PINAKA{vishvamitra_bala_atibala_one_syllable_unbound}",
  "proclamation": "The forbidden astra answers the broken invocation.",
  "result": "accepted",
  "weapon": "brahmastra"
}

A03_player ❯
```

Fix: use strong Fiat-Shamir and hash the full transcript, including the group parameters and the statement, so `c = H(domain, p, q, g, h, A, B, t1, t2)`. With `A1` in the hash, Brahmastra gets a different challenge and the `s + c*d` shift no longer verifies.

```
PINAKA{vishvamitra_bala_atibala_one_syllable_unbound}
```